There's no need to fill these in; just let them stand as they are while you're working on other specs. Here's what RSpec shows on the console when you have empty examples:

```
$ rspec
..**

Pending: (Failures listed here are expected and do not affect your suite's
status)

  1) A cup of coffee with milk is light in color
     # Not yet implemented
     # ./spec/coffee_spec.rb:34

  2) A cup of coffee with milk is cooler than 200 degrees Fahrenheit
     # Not yet implemented
     # ./spec/coffee_spec.rb:35

Finished in 0.00125 seconds (files took 0.07577 seconds to load)
4 examples, 0 failures, 2 pending
```

The two empty examples are marked with yellow asterisks on the progress bar and are listed as "Pending" in the output.

Marking Incomplete Work

When you're sketching out future work, you may actually have an idea of what you want the body of the spec to look like. It'd be nice to be able to mark some expectations as work in progress before you commit so that you're never committing a failing spec suite.

RSpec provides the `pending` method for this purpose. You can mark a spec as pending by adding the word `pending` anywhere inside the spec body, along with an explanation of why the test shouldn't pass yet. The location matters; any lines before the `pending` call will still be expected to pass. We typically add it at the top of the example:

02-running-specs/10/spec/coffee_spec.rb

```
it 'is light in color' do
  pending 'Color not implemented yet'
  expect(coffee.color).to be(:light)
end

it 'is cooler than 200 degrees Fahrenheit' do
  pending 'Temperature not implemented yet'
  expect(coffee.temperature).to be < 200.0
end
```

RSpec will run the body of the spec and print the failure so you can see it. But it won't mark the spec, or your overall suite, as failing: